

Documentation technique DATASHARE

A large rectangular graphic with a vertical gradient from light orange at the top to a darker red at the bottom. It contains the DataShare logo and a quote.

DataShare

« Nous gardons vos fichiers en toute sécurité »

Sommaire

1) Architecture de l'application (diagramme simple)	4
1.a) Vision globale	4
1) Frontend Angular (SPA)	4
2) Backend Spring Boot (API REST)	4
3) Base de données PostgreSQL	4
4) Stockage de fichiers (/uploads)	4
1.b) Interactions entre les briques	4
Frontend → Backend	4
Backend → PostgreSQL	5
Backend → Stockage local	5
Sécurité	5
1.c) Schéma simple (ASCII)	6
2) Choix technologiques justifiés (1–2 pages)	9
2.a) Langages	9
Java 17 (Backend)	9
TypeScript (Frontend)	9
2.b) Frameworks	9
Spring Boot 3	9
Angular 22	9
2.c) Base de données	10
PostgreSQL 18.4.2	10
2.d) Stockage des fichiers	10
Système local (/uploads)	10
2.e) Authentification et sécurité	10
JWT	10
BCrypt	10
2.f) Outils tiers	10
2.g) Outils de développement	11
3) Modèle de données	11
3.a) Description générale	11
users	11
files	11
3.b) Diagramme UML (PlantUML)	11
4) Documentation d'API	13
4.a) Authentification	13
POST /api/auth/register	13

POST /api/auth/login	13
POST /api/auth/refresh	13
4.b) Fichiers	13
POST /api/files/upload	14
GET /api/files/{id}	14
GET /api/files/download/{link}	14
GET /api/files/user/{userId}	14
DELETE /api/files/{id}	14
5) Sécurité et gestion des accès	15
5.a) Authentification	15
5.b) Permissions	15
5.c) Mesures de sécurité	15
6) Qualité, tests et maintenance	15
6.a) TESTING.md	15
6.b) SECURITY.md	16
6.c) PERF.md	16
6.d) MAINTENANCE.md	16
7) Processus d'installation et d'exécution	16
7.a) Prérequis	16
7.b) Installation	16
Création du package Jar pour le backend	16
Backend	16
Frontend	16
Base de Donnée	17
7.c) Variables d'environnement	17
8) Utilisation de l'IA dans le développement	17
8.a) Posture adoptée	17
8.b) Tâches confiées	17
8.c) Supervision	17
8.d) Apports	17
8.e) Limites	17

1) Architecture de l'application (diagramme simple)

1.a) Vision globale

L'application DataShare repose sur une architecture **full-stack découplée**, pensée pour être :

- **simple à maintenir,**
- **sécurisée,**
- **scalable,**
- **compatible avec les bonnes pratiques modernes.**

Elle se compose de quatre briques principales :

1) Frontend Angular (SPA)

- Interface utilisateur moderne et réactive.
- Gestion de l'authentification via JWT.
- Upload de fichiers via formulaire multipart.
- Routing interne sans rechargement de page (Single Page Application).
- Communication avec le backend via HTTP/HTTPS.

2) Backend Spring Boot (API REST)

- Expose des endpoints REST sécurisés.
- Gère la logique métier : upload, génération de lien, expiration.
- Vérifie les tokens JWT.
- Interagit avec PostgreSQL via JPA/Hibernate.
- Stocke les fichiers sur le système local.

3) Base de données PostgreSQL

- Stockage structuré des utilisateurs et des fichiers.
- Contraintes d'intégrité (PK, FK).
- Transactions ACID pour garantir la cohérence des données.

4) Stockage de fichiers (/uploads)

- Stockage physique (local) des fichiers envoyés.
- Séparation claire entre données (DB) et fichiers (FS).
- Chemin du fichier stocké dans la table `files`.
- Evolutif vers S3/MinIO.

1.b) Interactions entre les briques

Frontend → Backend

- Protocole : **HTTPS**
- Format : **JSON**
- Authentification : **JWT**

- Upload : **multipart/form-data**

Backend → PostgreSQL

- ORM : **Hibernate / JPA**
- Requêtes SQL générées automatiquement
- Transactions pour garantir la cohérence

Backend → Stockage local

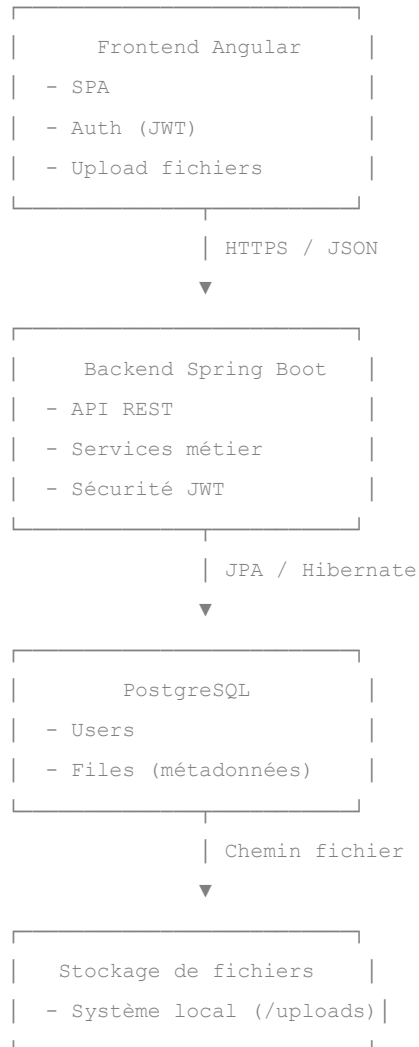
- Écriture des fichiers uploadés
- Lecture lors des téléchargements
- Vérification d'expiration et mot de passe

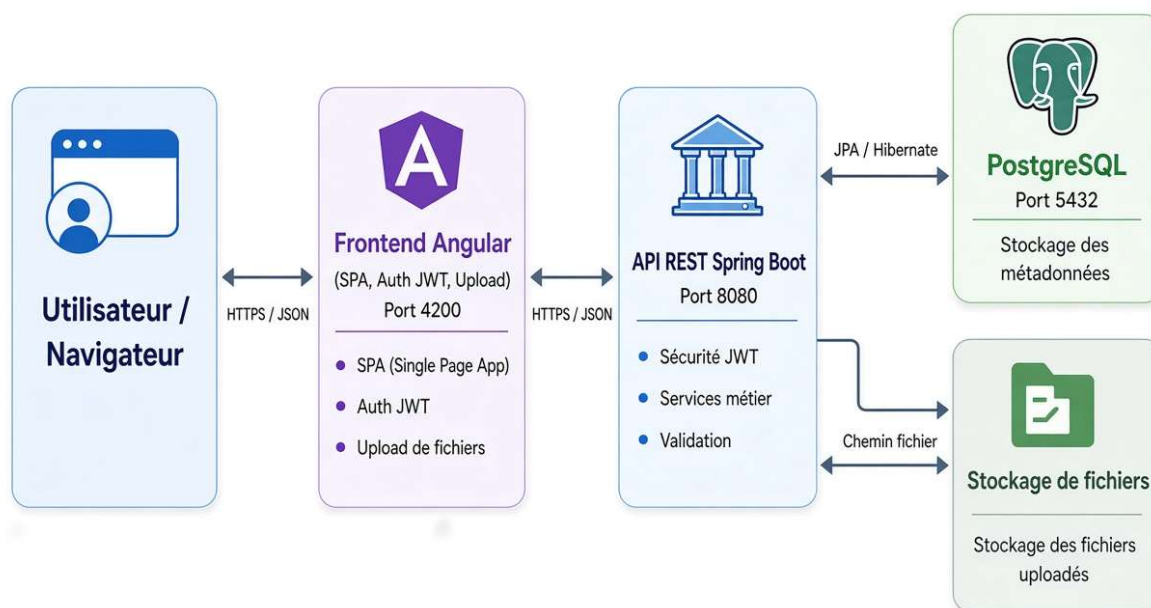
Sécurité

- JWT pour authentifier les utilisateurs
- BCrypt pour hasher les mots de passe
- Validation des entrées (Spring Validation)
- Limite de taille d'upload
- Expiration automatique des fichiers

1.c) Schéma simple (ASCII)

Code





Lecture détaillée du schéma d'architecture – DataShare

Le schéma décrit le parcours complet d'une requête dans l'application, de l'utilisateur jusqu'au stockage, avec des **flux bidirectionnels** (requête ↔ réponse).

1. Utilisateur / Navigateur

Rôle : point d'entrée humain.

L'utilisateur ouvre l'application dans son navigateur (Chrome, Firefox, Edge, etc.). Il interagit avec l'interface : connexion, upload, consultation de « Mes fichiers », téléchargement via un lien partagé.

- Communication en **HTTPS / JSON** avec le frontend
- Aucune logique métier côté navigateur pure : il affiche et envoie ce que l'application Angular lui demande

2. Frontend Angular – port 4200

Rôle : interface (SPA) et premier niveau d'interaction.

Élément	Signification
SPA	Une seule page chargée ; la navigation se fait sans rechargement complet
Auth JWT	Après login, le token JWT est stocké (ex. localStorage) et renvoyé dans les appels API
Upload de fichiers	Formulaire / sélection de fichier, envoi en multipart/form-data vers l'API

Flux ↔ API :

- **Aller** : requêtes HTTP (login, register, upload, liste, download, suppression)
- **Retour** : réponses JSON (token, métadonnées, erreurs) ou fichier binaire

En développement : <http://localhost:4200>.

3. API REST Spring Boot – port 8080

Rôle : cœur métier et sécurité.

Élément	Signification
Sécurité JWT	Filtre qui valide le token ; certains endpoints restent publics (metadata, download, upload anonyme)
Services métier	Upload, historique, suppression, tags, expiration, partage
Validation	Taille max (1 Go), extensions interdites, règles mot de passe, durée d'expiration

C'est le **seul** composant qui parle à la base et au disque. Le frontend ne touche jamais PostgreSQL ni le dossier uploads/ directement.

En développement : `http://localhost:8080`.

4. PostgreSQL – port 5432

Rôle : stockage des **métadonnées**, pas des binaires.

Exemples de données :

- utilisateurs (email, hash du mot de passe)
- fichiers (nom, taille, type, token de download, expiration, ownerId, tags, hash du mot de passe fichier)

Flux ↔ **API (JPA / Hibernate)** :

- **Écriture** : inscription, upload, tags, etc.
- **Lecture** : liste « Mes fichiers », metadata avant download, contrôles d'autorisation

Le fichier lui-même n'est **pas** dans PostgreSQL.

5. Stockage de fichiers

Rôle : conservation des **binaires** uploadés (dossier local type uploads/).

Flux ↔ **API (chemin fichier)** :

- **Écriture** : lors de l'upload
- **Lecture** : lors du téléchargement via le token

Pas de port réseau : accès disque depuis le process Spring Boot. En production, on peut remplacer ce dossier par un stockage objet (S3, MinIO) sans changer le reste du schéma.

Synthèse des échanges

text

Utilisateur ↔ Angular (:4200) ↔ Spring Boot (:8080) ↔ PostgreSQL (:5432)

↑

Stockage fichiers (disque)

Parcours typique	Chemin dans le schéma
Connexion	Navigateur → Angular → API → PostgreSQL (vérif user) → JWT renvoyé
Upload	Navigateur → Angular → API → disque + métadonnées PostgreSQL
Liste « Mes fichiers »	Angular → API → PostgreSQL → JSON affiché
Téléchargement par lien	Angular (page download) → API (metadata + fichier) → PostgreSQL + disque
Suppression	Angular → API → suppression disque + ligne PostgreSQL

Points d'architecture à retenir

1. **Séparation des responsabilités** : UI / API / données / fichiers bien isolés.
2. **Bidirectionnalité** : chaque flèche représente un aller-retour (pas un flux « à sens unique »).
3. **Sécurité centralisée** dans l'API (JWT, validation, propriété des fichiers).

4. **Scalabilité** : on peut faire évoluer le stockage fichiers ou la BDD sans refondre le frontend.

2) Choix technologiques justifiés (1–2 pages)

2.a) Langages

Java 17 (Backend)

- Support long terme (LTS).
- Excellent pour les API REST sécurisées.
- Intégration native avec Spring Boot.
- Gestion avancée des threads, sécurité, performance.
- Parfait pour un projet professionnel.

TypeScript (Frontend)

- Typage fort → réduction des bugs.
- Meilleure lisibilité et maintenabilité.
- Standard moderne pour Angular.
- Permet une architecture claire et scalable.

2.b) Frameworks

Spring Boot 3

- Démarrage rapide (auto-configuration).
- Spring Security pour JWT.
- Spring Web pour API REST.
- Spring Validation pour sécuriser les entrées.
- JPA/Hibernate pour PostgreSQL.
- Structure MVC claire.

Angular 22

- Architecture modulaire (modules, services, composants).
- CLI puissante pour générer du code.
- Routing intégré.
- Interceptors pour gérer JWT.
- Formulaires réactifs.
- Idéal pour une SPA professionnelle.

2.c) Base de données

PostgreSQL 18.4.2

- Base SQL robuste et fiable.
- Support JSONB si besoin d'évolution.
- Transactions ACID.
- Parfait pour relations simples (User → Files).
- Compatible avec Hibernate/JPA.

2.d) Stockage des fichiers

Système local (/uploads)

- Simple à mettre en place.
- Performant pour un MVP.
- Pas de dépendance externe.
- Séparation claire entre données et fichiers.
- Compatible avec les spécifications DataShare.

2.e) Authentification et sécurité

JWT

- Authentification stateless.
- Facile à intégrer dans Angular via interceptor.
- Sécurisation des endpoints backend.

BCrypt

- Hashage sécurisé des mots de passe.
- Standard de l'industrie.

2.f) Outils tiers

Élément	Technologie choisie	Alternatives	Justification
Hébergement	Docker	VM, Kubernetes	Reproductible, simple
CI/CD	GitHub Actions	GitLab CI	Intégration fluide
Documentation API	Swagger	Postman	Standard REST
Dépendances	Maven / npm	Gradle / yarn	Standards
IDE	IntelliJ / VS Code	Eclipse	Productivité

2.g) Outils de développement

- Git & GitHub pour versioning.
- Springdoc OpenAPI (Swagger) — documentation interactive
- ESLint + Prettier pour qualité du code Angular.
- JUnit + MockMvc +Jacoco pour tests backend.
- Cypress pour tests end-to-end.
- Swagger pour tests API
- Postman pour tests API.

3) Modèle de données

3.a) Description générale

Le modèle repose sur **2 tables** :

users

- Stocke les comptes utilisateurs.
- Contient email, mot de passe hashé, date de création.
- Relation 1-N avec les fichiers.

files

- Stocke les métadonnées des fichiers.
- Contient le chemin physique, le lien unique, l'expiration, le mot de passe optionnel.
- Peut appartenir à un utilisateur ou être anonyme.

3.b) Diagramme UML (PlantUML)

```
plantuml
@startuml

entity User {
    * id : UUID
    --
    email : String
    passwordHash : String
    createdAt : Timestamp
}

entity File {
    * id : UUID
    --
    filename : String
    size : Long
    contentType : String
    storagePath : String
}
```

```

downloadLink : String
passwordHash : String?
expirationDate : Date
createdAt : Timestamp
ownerId : UUID?
tags : String?
}

```

```

User ||--o{ File : owns

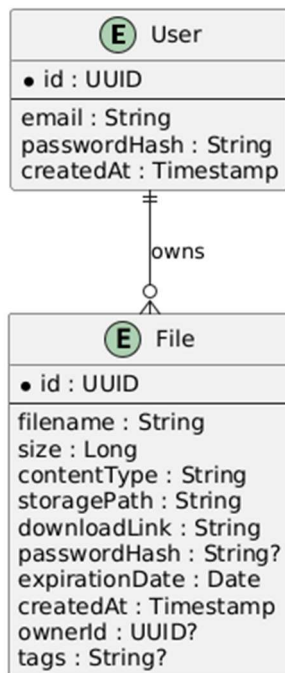
```

```

@enduml

```

Schéma du Modele de donnée : DATASHARE



4) Documentation d'API

4.a) Authentification

POST /api/auth/register

Créer un utilisateur.

Request

```
json
{
  "email": "user@example.com",
  "password": "secret"
}
```

Response

```
json
{
  "id": "uuid",
  "email": "user@example.com"
}
```

POST /api/auth/login

Retourne un JWT.

Response

```
json
{
  "token": "jwt-token",
  "userId": "uuid"
}
```

POST /api/auth/refresh

Renouvelle le token.

4.b) Fichiers

POST /api/files/upload

Upload multipart.

Payload

- file
- password (optionnel)
- expirationDate (optionnel)
- tags (optionnel)

Response

```
json
{
  "id": "uuid",
  "downloadLink": "abc123",
  "ownerId": "uuid | null"
}
```

GET /api/files/{id}

Téléchargement via ID.

GET /api/files/download/{link}

Téléchargement via lien unique.

GET /api/files/user/{userId}

Liste des fichiers d'un utilisateur.

DELETE /api/files/{id}

Suppression.

PUT /api/files/{id}/tags

Application des tags

POST /api/files/upload/anonymous

Téléversement anonyme

POST /api/share

Partage de fichiers

POST /api/auth/register

Création de nouveaux utilisateurs

POST /api/auth/login

Connexion des utilisateurs existants

GET /api/files/history

Liste les fichiers existants de l'utilisateur

5) Sécurité et gestion des accès

5.a) Authentification

- JWT signé avec secret.
- Stocké côté client dans localStorage.
- Vérifié par Spring Security.

5.b) Permissions

- Un utilisateur ne peut accéder qu'à ses fichiers.
- Les fichiers anonymes sont accessibles via lien unique.
- Expiration automatique.

5.c) Mesures de sécurité

- Hashage BCrypt.
- Validation des entrées.
- Limite d'upload.
- HTTPS recommandé.
- Protection contre injection SQL via JPA.

6) Qualité, tests et maintenance

6.a) TESTING.md

- Tests unitaires backend (JUnit).
- Tests E2E frontend (Cypress).
- Tests API (Swagger)
- Tests API (Postman).
- Critères d'acceptation par US.

6.b) SECURITY.md

- Scan de dépendances.
- Analyse des risques.
- Décisions appliquées (ex : limite d'upload).

6.c) PERF.md

- Test de performance endpoint upload.
- Analyse des temps de réponse.
- Budget front (taille bundle Angular).
- Logs navigateur + serveur.

6.d) MAINTENANCE.md

- Procédures de mise à jour.
- Intégration automatisée avec Dependabot et Renovate
- Fréquence.
- Risques (ex : corruption fichier).

7) Processus d'installation et d'exécution

7.aPrérequis

- Node 18+ (24.16)
- Angular CLI 22
- Java 17
- PostgreSQL 18.4.2
- Docker (optionnel)

7.b) Installation

Création du package Jar pour le backend

https://start.spring.io/?utm_source=copilot.com

Backend

Code

```
mvn clean install
mvn spring-boot:run
```

Frontend

Code


```
npm install
ng serve
```

Base de Donnée

Téléchargement et Installation du client PostgreSQL sur le poste de travail:

<https://www.postgresql.org/download/windows/>

7.c) Variables d'environnement

- DB_URL
- DB_USER
- DB_PASSWORD
- JWT_SECRET

8) Utilisation de l'IA dans le développement

8.a) Posture adoptée

- IA utilisée comme binôme technique.
- Assistance rédactionnelle et technique.

8.b) Tâches confiées

- Génération UML.
- Documentation API.
- Structuration Angular/Spring Boot.
- Conseils d'architecture.

8.c) Supervision

- Revue de code manuelle.
- Ajustements sécurité.
- Optimisation performance.

8.d) Apports

- Gain de temps.
- Inspiration.
- Structuration claire.

8.e) Limites

- Vérification nécessaire.

- Ajustements techniques.